LESSON GUIDE

Technology Plus

Unit 5: Software

3.1.1 – OS File Systems

Lesson Overview:

In this lesson, students will explore how an operating system's filesystem organizes, describes, and protects data – connecting everyday file interactions to underlying concepts like metadata, compression, encryption, file types, and extensions.

Students will:

- Describe the role of a filesystem in organizing and managing data.
- Differentiate between common filesystem characteristics, including compression, encryption, file types, extensions, and metadata.
- Compare the features of different filesystems used by modern operating systems.

Guiding Question: How does an operating system's filesystem organize, protect, and describe the files we use every day?

Suggested Grade Levels: 8-12

Technology Needed: Yes

Approximate Time Required: 60-75 minutes

Standards:

CompTIA Tech+ FC0-U71 Objective

- 3.1 – Identify the components of an OS.
  - Filesystem characteristics
    - Compression
    - Encryption
    - Types and extensions

This content is based upon work supported by the US Department of Homeland Security's Cybersecurity & Infrastructure Security Agency under the Cybersecurity Education Training and Assistance Program (CETAP).

Lesson Background

Background Information:

Filesystems are one of those topics students use every single day without realizing it. Every time they save a document, send a photo, or plug in a flash drive, the OS's filesystem is quietly deciding how to store it, what information to keep about it, and who gets to open it. Most students think of files only in terms of icons on a desktop, so this lesson pulls back the curtain to show the hidden structure that makes those icons possible. Understanding filesystem characteristics – like compression, encryption, and file types – gives students a foundation for troubleshooting storage issues, protecting data, and navigating different operating systems without getting lost

Materials Needed:

Materials per Class	· Lesson Guide · Lesson Slides 3.1.1 – OS File Systems · Lab Slides 3.1.1 – Filesystem Detective
Materials per Group	· File container (shoebox/tote/large envelope) · Assorted files (docs, pictures, CDs/DVDs, flash drive, random object)
Materials per Student	· Student Notes 3.1.1 – OS File Systems · Student Handout 3.1.1 – OS File Systems · Assessment 3.1.1 – OS File Systems (optional)

Lesson Background

Cyber Terms:

Term	Definition
filesystem	the structure an operating system uses to organize, store, and manage files on a storage device
compression	the process of reducing file size to save space or speed up transfers
lossless compression	a type of compression that reduces file size without losing any data, allowing the file to be fully restored
lossy compression	a type of compression that permanently removes some data to reduce file size
encryption	a method of protecting data by converting it into unreadable code unless a decryption key is used
symmetric encryption	encryption that uses the same key to encrypt and decrypt data
asymmetric encryption	encryption that uses two keys – a public key to encrypt and a private key to decrypt
journaling	a feature of some modern filesystems that helps protect data in case of a crash by recording file changes in a special log called a journal
file type	a label that describes the kind of data in a file, such as text, image, video, or executable
file extension	the part of a file name (after the dot) that tells the OS what type of file it is and which program should open it
metadata	extra information stored with a file, such as its name, size, creation date, and attributes like read-only or hidden

3.1.1 – OS File Systems

Guiding Question: How does an operating system's filesystem organize, protect, and describe the files we use every day?

Lesson Launch (15-20 minutes)

Contents Chaos

Students will investigate an assortment of unsorted "files" to see what they can learn without direct access, then bring order to the chaos – just like an OS filesystem.

Step 1 – The Metadata Stage

- Prepare several mystery file containers for student groups using the materials listed above.
- Distribute those containers to students but tell them not to open the containers.
- Distribute the Student Handout 3.1.1 – OS File Systems.
- Have students explore the mystery container without opening it. They can measure the dimensions of the box, comment on its weight, make note of the date and time that they were given the box, and any other observations they can make without opening the box.
- Have them record their observations on the student handout.

Step 2 – Access Granted

- Allow students to open the container and observe its contents. They can move stuff around, but don't allow them to remove anything from the container just yet – we don't want them to accidentally start sorting the items.
- Have students record what they find in the container on the student handout.

Step 3 – Organization Stage

- Ask the students to sort the container's contents into categories. They can decide on structure, but they must be ready to explain their reasoning.
- Have them record their observations on the student handout.

Step 4 – Reflection

- Use the following questions to debrief with your students:
    - How did the first stage limit what you knew?
    - What "metadata" was available before you opened the box? (If students are unfamiliar with this term, connect it to everyday examples: the track length and artist info on a music file,

the author and date on a Google Doc, or the caller ID info before you pick up the phone. Metadata is the "about" info that helps you understand what you're dealing with before you open or use it.)
    - What did organizing the items do for your ability to find and use them?
    - How is this like an operating system's filesystem?
- **Transition to content:** Highlight how the "mystery box" activity mirrors a filesystem's job – providing information about stored items, keeping them organized, and making them retrievable later.

Fast Facts on Filesystems (15-20 minutes)

What a Filesystem Does

- A ***filesystem*** acts as the OS's "roadmap" for all stored data.
- Without a filesystem, the disk is just raw binary data with no organization.
- Responsibilities include:
    - Organizing data into a directory structure (flat vs. hierarchical)
    - Managing permissions and security
    - Tracking metadata for files and folders
    - Providing features like compression, encryption, and journaling
- Hierarchical = like a filing cabinet with folders inside folders.
- Flat = pretty rare in modern operating systems, but minimal OS-like environments on certain microcontrollers (like the BBC micro:bit) use flat filesystems internally – they store everything in one directory without subfolders.
- *Allocation methods* influence performance and space use:
    - **Contiguous allocation** – fastest read performance, prone to fragmentation
    - **Linked allocation** – flexible storage but slower access
    - **Indexed allocation** – uses an index block; common in modern filesystems
- **Ask:** "Why does my flash drive work on my computer but not my friend's?"
  Students may say, "The filesystem might not be recognized by the friend's OS. Example: Windows can't read Linux's ext4 without special software."

Compression

- ***Compression*** saves disk space and can speed up file transfers.
- ***Lossless:*** perfect restoration (ZIP, PNG, GZIP)
- ***Lossy:*** smaller size but data permanently removed (JPEG, MP3, MP4)
- Common algorithms: Huffman coding, Lempel–Ziv–Welch (LZW), Run-Length Encoding (RLE)
- Already-compressed files gain little benefit from additional compression
- *Historical note:* Compression was once vital due to limited storage; Windows still offers

"Compress this drive to save space" for legacy reasons

- **Ask:** "Why doesn't zipping a JPEG make it smaller?"

  Students may say, "JPEG is already compressed; re-compressing offers little to no reduction."
- **Demo idea:** Compare zipped .txt vs. zipped .jpg file sizes.

Encryption

- ***Encryption*** transforms data so only someone with the key can read it.
- ***Symmetric encryption:*** one key for both encryption/decryption (Advanced Encryption Standard – AES, Data Encryption Standard – DES).
- ***Asymmetric encryption:*** key pair – public for encrypting, private for decrypting (Rivest-Shamir-Adleman – RSA, Elliptic Curve Cryptography – ECC).
- Common applications:
  - File-level encryption (Encrypting File System – EFS)
  - Full-disk encryption (BitLocker, FileVault)
  - Transmission security (Transport Layer Security – TLS/HTTPS – Hypertext Transfer Protocol Secure) – network encryption (TLS) in HTTPS is the same principle but for data in transit.
- **Ask:** "If I take a BitLocker drive and plug it into another computer, will it work?"

  Students may say, "Not without the decryption key or password – encryption protects the data outside its original environment."

Filesystem Types

Filesystem	Primary OS	Notable Features
New Technology File System (NTFS)	Windows	Modern standards; supports permissions, encryption, and journaling
File Allocation Table 32-bit (FAT32)	Windows/ Removable drives	Older; limited to 4 GB files, no journaling
Fourth Extended File System (ext4)	Linux	Supports large volumes and journaling
Extended Filesystem (XFS)	Linux	High performance; supports large partitions
Apple File System (APFS)	macOS	Optimized for SSDs; default for Apple systems
Universal Disk Format (UDF)	Media discs	Used for DVDs, Blu-rays, etc.

Historical context: FAT12/FAT16 were early DOS/Windows file systems–small max partitions, no security, no journaling.

Tech Tip: OS must recognize filesystem to read it.

Access Control Lists (ACLs) & Permissions

- Define which users/groups can read, write, execute, or modify files.
- Can be inherited from folders or set individually.
- Granular controls make it possible to tightly manage data security.

Journaling

- ***Journaling*** is when a filesystem writes changes to a log (journal) before committing them.
- Protects against corruption from crashes or power loss.
- Allows recovery or rollback to a stable state.
- Non-journaled filesystems are more likely to lose data in a crash.

File Types & Extensions

- ***File type*** describes content (text, image, video, etc.).
- ***File extension*** tells OS which program to use to open it.
- Historical 8.3 rule: up to 8-character filename + 3-character extension (e.g., report98. doc).
- Modern systems support 255 characters and Unicode but still restrict certain symbols (\ / : * ? " < > |).
- Extensions are critical for opening files correctly.
- **Demo idea:** Change a .txt to .jpg and try to open it.
- ***Metadata*** includes name, size, timestamps, attributes (read-only, hidden).

Common File Types

Type	Examples	Use
Text/Docs	.txt, .docx, .pdf	Written content and reports
Images	.jpg, .png, .gif	Pictures, screenshots
Media	.mp3, .mp4, .wav	Music, videos, audio clips
Executables	.exe, .app, .elf	Runs programs (Windows, macOS, Linux)
Compressed Archives	.zip, .rar, .7z	Bundled/compressed files
Web Files	.html, .css, .js	Website content

Lab: Filesystem Detectives (25-30 minutes)

- Display Lab Slides 3.1.1 – Filesystem Detectives, have students record their answers in the handout.

Part A – Metadata

- Students will use file properties (Windows) or the **`ls -lh, file`** and **`stat`** commands (Ubuntu) to gather file metadata such as size, type, and timestamps.
- **Teacher tip:** Point out how much they can learn without opening the file. Tie this back to the "Contents Chaos" activity and the concept of metadata.
- **Ask:** "What kind of details did you get that might help you decide what's in this file?"

  Students may say, "File size, file type/extension, dates (created, modified, accessed), permissions, location in the directory."
- **Possible student roadblock:** Some students may confuse "file type" with "file extension." Remind them that the OS determines type from file contents, not just the name.

Part B – Compression

- Students will zip a plain text file and compare sizes.
- In Ubuntu, they'll see a "deflated" percentage – encourage them to compare compression results for gatsby.txt vs. art-panda.txt.
- **Teacher tip:** Emphasize redundancy in data (repeated characters, common words) as the reason for differences in compression ratios.
- **Ask:** "If a file barley changes in size after compression, what might that suggest about the file?"

  Students may say, "The file is already compressed," or "The file has little to no repeating patterns for the algorithms to remove," or "The files might be encrypted – encrypted data often looks random and doesn't compress well," or "The file could be small enough that compression isn't really necessary."
- **Optional extension:** Have students open the compressed archive to confirm that all original data is still intact.

Part C – Encryption

- In Windows, students encrypt a file using the "Encrypt contents to secure data" option. Discuss how this depends on the user account and OS.
- In Ubuntu, they use **`zip -e`** (or **`--encrypt`**) to password-protect a file and see that the same account still needs the password to open it.
- **Teacher tip:** This is a great spot to contrast *file-level* encryption vs. *archive* encryption.
- **Ask:** "In Ubuntu, you had to type a password to open the file – in Windows, you didn't. What does that tell you about how each system handles encryption?"

Students may say, "Window file encryption (EFS) ties access to the user account – if you're logged in as the user who encrypted the file, no password is necessary. Ubuntu's zip -e encryption is tied to the file itself – every time it's opened, the password is required regardless of who's logged in."

Putting It All Together

- Which of the three techniques used in this lab – metadata inspection, compression, or encryption – do you think is most important for protecting files? Why?

  Answers will vary; students should connect their choice to a clear reason (e.g., encryption protects confidentiality, compression saves storage, metadata helps track file history).

- How can metadata be helpful when investigating suspicious or unexpected files?

  It can reveal creation/modification dates, file size, and type, which may expose tampering, mismatched extensions, or unusual activity.

- What's one real-world scenario where compressing a file before sharing it would be beneficial?

  Answers may vary; but students could mention scenarios like sending large text documents over email, archiving logs, preparing files for slower network transfer, etc.

- Highlight the big idea: **Filesystems don't just store data — they describe, compress, protect, and organize it.**

Putting It All Together (5 minutes)

- Why does an operating system need a filesystem?

  Without a filesystem, the OS would have no way to organize or track files – data would just be raw 1s and 0s on the storage device. The filesystem provides structure, metadata, and access rules so files can be stored, found, and managed.

- How is compression different from encryption?

  Compression reduces the file size to save space or speed up transfers, while encryption changes data into unreadable code to protect it from unauthorized access. Compression focuses on efficiency; encryption focuses on security.

- Why might a file created on one system not open on another?

  The second system may not recognize the filesystem type (e.g., Windows cannot read Linux's ext4 without extra software) or may not support the same file permissions or encryption methods.

- What's one real-world example of metadata that you use without thinking about it?

  Caller ID on a phone shows the number, name, and time of the call before answering – information about the call, not the call itself. Similarly, a music player showing track length and artist info is metadata about the audio file. Accept all reasonable answers.

- Every file you open, share, or save carries hidden details – how it's stored, who can access it, and what it reveals without opening it. The filesystem manages that story. And now you know how to read it.